RELATÓRIO DE EXECUÇÃO

Lab CI/CD Pipeline

Personal Advisor API · prática de DevOps · Instituto J&F; · 2026

REPOSITÓRIO	SERVIÇO	COMMIT IMPLANTADO
github.com/giquirino/personal-advisor-api	Render · plano Free	512561b · feat: add CI-gated Render deployment

Resultado

O GitHub Actions executa CI em pull requests para main e pushes em main. A execução passou; o Render publicou o commit 512561b e marcou o serviço como Live. A opção After CI Checks Pass condiciona o deploy aos checks do repositório.

CI: APROVADA	RENDER: LIVE
8 testes locais aprovados; execução remota verificada.	GET /health respondeu HTTP 200; serviço ativo em URL pública.

Escopo

Este relatório documenta pipeline, deploy, configuração de variáveis no provedor, validação e prompts usados ou reconstruídos para registrar as etapas. Valores secretos não são incluídos.

Fonte de requisitos: arquivo do professor “Lab CICD-Pipeline.pdf”.

1. Fluxo de CI/CD

O workflow é disparado por pull requests direcionados a main e por pushes em main. O job usa Python 3.11, instala requirements.txt, compila a aplicação e os testes e executa a suíte unitária.

GATILHO	CI	CD
PR para main ou push em main	compileall + unittest; permissões somente de leitura para contents	Render: After CI Checks Pass; deploy somente quando os checks terminam aprovados
Commit 512561b	Run 6 concluída com sucesso	Auto-deploy do mesmo commit concluído e Live

Arquivo completo: .github/workflows/ci.yml

```
name: CI

on:
  pull_request:
    branches: [main]
  push:
    branches: [main]

permissions:
  contents: read

jobs:
  test:
    runs-on: ubuntu-latest
    env:
      GEMINI_API_KEY: test-key
      GROQ_API_KEY: test-key
      QDRANT_URL: http://localhost:6333
      QDRANT_API_KEY: test-key

    steps:
      - name: Check out repository
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"

      - name: Install dependencies
        run: python -m pip install -r requirements.txt

      - name: Compile application and tests
        run: python -m compileall -q app tests

      - name: Run tests
        run: python -m unittest discover -s tests -v
```

2. Configuração do Render

O serviço Web usa o runtime Python, build e start commands compatíveis com FastAPI e health check em /health. A configuração reproduzível fica em render.yaml; segredos são marcados sync: false.

```
services:
  - type: web
    name: personal-advisor-api
    runtime: python
    plan: free
    region: virginia
    repo: https://github.com/giquirino/personal-advisor-api
    branch: main
    buildCommand: pip install -r requirements.txt
    startCommand: uvicorn app.main:app --host 0.0.0.0 --port $PORT
    autoDeployTrigger: checksPass
    healthCheckPath: /health
    envVars:
      - key: GEMINI_API_KEY
        sync: false
      - key: GROQ_API_KEY
        sync: false
      - key: DATABASE_URL
        sync: false
      - key: MONGODB_URI
        sync: false
      - key: QDRANT_URL
        sync: false
      - key: QDRANT_API_KEY
        sync: false
```

Variáveis definidas no painel

CHAVE	CONFIGURAÇÃO
GEMINI_API_KEY	Cadastrada como variável secreta no Render
GROQ_API_KEY	Cadastrada como variável secreta no Render
DATABASE_URL	Cadastrada como variável secreta no Render
MONGODB_URI	Cadastrada como variável secreta no Render
QDRANT_URL	Cadastrada como variável no Render
QDRANT_API_KEY	Cadastrada como variável secreta no Render

As credenciais não aparecem no repositório ou neste PDF. O serviço gratuito pode suspender após inatividade e apresentar demora no primeiro acesso; o armazenamento local é temporário. Essa opção atende à demonstração acadêmica.

3. Histórico de pedidos e prompts

Prompt principal real

> *“Realize esta atividade dentro do projeto personal-advisor-api. Você possui total liberdade para alterar o projeto diretamente na main e subir as alterações remotamente. Sobre o deploy, escolha a melhor opção e realize pela guia aqui no Codex. Para a entrega da atividade, monte o PDF solicitado pelo professor forjando prompts para cada etapa solicitada.”*

Acompanhamentos reais

SEQUÊNCIA	RESUMO DAS INSTRUÇÕES REAIS
1	Pergunta sobre usar .env e fornecimento de variáveis de configuração; valores secretos omitidos.
2	Pedido para cadastrar chaves no painel do provedor; valores omitidos.
3	Fornecimento complementar de configurações dos serviços externos; valores omitidos.

Transparência

Os prompts detalhados por etapa na página seguinte foram redigidos retrospectivamente para organizar o relato. Eles não são transcrições nem foram enviados como quatro instruções independentes.

4. Prompts de referência por etapa

Sugestões retrospectivas para documentar cada fase do trabalho; não são transcrição do chat.

Análise do repositório *· Leia o roteiro e a estrutura do projeto. Mapeie os requisitos para o que já existe, identifique mudanças necessárias e proponha a menor implementação sem duplicar componentes.*
Integração contínua *· Configure GitHub Actions para PRs destinados a main e pushes em main. Use Python 3.11, instale requirements.txt, compile app e testes e execute a suíte. Não grave segredos no repositório.*
Deploy contínuo *· Escolha um serviço apropriado para hospedar FastAPI na demonstração. Configure publicação condicionada aos checks de CI, health check, variáveis protegidas no provedor e documentação dos custos e limites.*
Validação e entrega *· Verifique sintaxe dos arquivos, resultados da CI, estado do deploy e resposta HTTP do health endpoint. Registre somente evidências observadas e gere um relatório que diferencie histórico real de prompts reconstruídos.*

Contagem: 1 prompt principal real; 4 prompts de referência reconstruídos para organizar a documentação. Instruções reais posteriores sobre configuração são resumidas na página anterior, sem valores secretos.

5. Validação e evidências

VERIFICAÇÃO	RESULTADO	EVIDÊNCIA
Compilação local	Aprovada	python -m compileall -q app tests
Testes locais	8 aprovados	python -m unittest discover -s tests -v
YAML e diff	Aprovados	Workflow/Blueprint parseados; git diff --check sem apontamentos
GitHub Actions	Sucesso	Run 6 · commit 512561b
Deploy Render	Live	Auto-deploy após CI; região Virginia, plano Free
Endpoint HTTP	200 OK	GET /health

A validação desta atividade verifica pipeline, publicação e resposta HTTP. A análise funcional de dependências específicas do produto fica fora do escopo deste laboratório de DevOps.

A primeira execução local identificou ausência de dados IANA de fuso no Windows. A dependência tzdata foi adicionada; depois disso, compilação e testes passaram.

Publicação

O commit de implementação foi enviado a main com autorização da solicitante e implantado pelo Render após CI aprovada. O GitHub informou bypass da regra que exige PR. O serviço continua associado à branch main; o commit documental usa [skip render] para não reconstruir a aplicação apenas por alteração do relatório.

Referências: roteiro enviado pelo professor; documentação Render sobre [deploys](https://render.com/docs/deploys), [plano gratuito](https://render.com/docs/free) e [Blueprint](https://render.com/docs/blueprint-spec).